

Practical No.2

Aim: Implement Monoalphabetic Cipher encryption and decryption

Code:

```
import * as readline from "readline";
import { stdin as input, stdout as output } from "process";

function question(rl: readline.Interface, prompt: string): Promise<string> {
    return new Promise((resolve) => rl.question(prompt, resolve));
}

const ALPHABET = "abcdefghijklmnopqrstuvwxyz";
const ALPHABET_UPPER = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

/** Build substitution map: plain letter -> cipher letter (lowercase key)
function buildEncryptMap(key: string): Map<string, string> {
    const keyLower = key.toLowerCase();
    const map = new Map<string, string>();
    for (let i = 0; i < ALPHABET.length; i++) {
        map.set(ALPHABET[i], keyLower[i]);
        map.set(ALPHABET_UPPER[i], keyLower[i].toUpperCase());
    }
    return map;
}

/** Build reverse map: cipher letter -> plain letter. */
function buildDecryptMap(key: string): Map<string, string> {
    const keyLower = key.toLowerCase();
    const map = new Map<string, string>();
    for (let i = 0; i < ALPHABET.length; i++) {
        map.set(keyLower[i], ALPHABET[i]);
        map.set(keyLower[i].toUpperCase(), ALPHABET_UPPER[i]);
    }
    return map;
}

function monoalphabeticEncrypt(plainText: string, key: string): string {
    const map = buildEncryptMap(key);
    return plainText
        .split("")
        .map((char) => map.get(char) ?? char)
        .join("");
}

function monoalphabeticDecrypt(cipherText: string, key: string): string {
    const map = buildDecryptMap(key);
```

```
    return cipherText
      .split("")
      .map((char) => map.get(char) ?? char)
      .join("");
  }

function isValidKey(key: string): boolean {
  const k = key.replace(/\s/g, "").toLowerCase();
  if (k.length !== 26) return false;
  const seen = new Set<string>();
  for (const c of k) {
    if (c < "a" || c > "z") return false;
    if (seen.has(c)) return false;
    seen.add(c);
  }
  return true;
}

async function main(): Promise<void> {
  const rl = readline.createInterface({ input, output });

  const operation = await question(rl, "Encrypt or Decrypt? (e/d): ");
  const normalizedOp = operation.trim().toLowerCase();
  if (normalizedOp !== "e" && normalizedOp !== "d") {
    console.error("Invalid choice. Use 'e' for encrypt or 'd' for decrypt");
    rl.close();
    process.exit(1);
  }

  const textPrompt =
    normalizedOp === "e" ? "Enter plaintext: " : "Enter ciphertext: ";
  const text = (await question(rl, textPrompt)).trim();

  const keyPrompt =
    "Enter key (26 unique letters, e.g. qwertyuiopasdfghjklzxcvbnm): ";
  const keyInput = (await question(rl, keyPrompt)).trim().replace(/\s/g, "");

  if (!isValidKey(keyInput)) {
    console.error(
      "Invalid key. Use exactly 26 unique letters (a-z).");
    rl.close();
    process.exit(1);
  }

  rl.close();

  if (normalizedOp === "e") {
```

```
        console.log("Encrypted:", monoalphabeticEncrypt(text, keyInput));
    } else {
        console.log("Decrypted:", monoalphabeticDecrypt(text, keyInput));
    }
}

main().catch((err) => {
    console.error(err);
    process.exit(1);
});
```

Output:

practical2-output.png